📱 THE FACE

Frontend & Mobile Lead — Muhammad Rizki

ERRNOR Team · Aplikasi LUNAS · Mobile Build Guide 2025

📌 BRIEF — SIAPA LU DI ERRNOR?

Lu bukan tukang ketik kode. Lu adalah Algojo Visual ERRNOR. Tugas lu satu: mengubah desain buatan Nabil jadi layar HP yang bisa di-scroll, di-klik, dan memanjakan mata juri. Layar yang lu buat adalah WAJAH pertama yang dilihat user dan juri hackathon. Tanpa wajah yang mulus, semua mesin backend dan kecerdasan AI dari Ibnul tidak akan pernah terlihat. Lu adalah garis depan — buat juri jatuh cinta di 3 detik pertama.

🏗️ KONTEKS PROYEK — APA YANG LU BANGUN?

Pahami dulu "wajah" seperti apa yang harus lu lukis. Frontend bukan hanya UI — ini pengalaman emosional user saat berinteraksi dengan LUNAS.

LUNAS adalah asisten keuangan otonom dengan 3 lapisan sistem. Frontend (kamu) adalah lapisan yang bersentuhan langsung dengan user. Lu yang menentukan apakah user merasa **"WOW, aplikasi ini keren"** atau langsung uninstall.

📱 FRONTEND	⚙️ BACKEND	🧠 AI ENGINE
Muhammad Rizki — INI KAMU	Arienal Zacky Irfan	Ibnul Jawzy
Flutter · Dart	FastAPI · Python · PostgreSQL	Gemini · LangChain · RAG
Dashboard Net-WorthPush Notification Nudging1-Tap Approve2FA Biometrik	REST API EngineAlgoritma AvalancheDatabase ManagerOrkestrasi SNAP BI	Behavioral NudgingAnalisis TransaksiKnowledge Base OJKVector Embeddings

💡 MENTAL MODEL — POSISI LU DI EKOSISTEM LUNAS

Bayangkan LUNAS kayak restoran mewah. Kamu (Frontend) adalah interior designer dan maître d' — yang bikin customer terpesona begitu masuk pintu. Arienal (Backend) adalah dapur yang masak pesanannya. Ibnul (AI) adalah chef spesialis analitik. Nabil (Hustler) adalah pemilik yang nyewain gedung dan ngurusin ijin usaha. Tanpa tampilan yang mewah dan nyaman, tidak ada yang mau masuk ke restoran ini.

🔧 ARSENAL LU — SEMUA TOOLS YANG AKAN DIPAKAI

Kenali semua senjata sebelum masuk medan perang. Ini tech stack lengkap frontend LUNAS dari awal hingga demo.

Tool	Fungsi & Peran di LUNAS	Platform	Cara Install
Flutter 3.x	Framework utama untuk membangun UI cross-platform (Android & iOS) dari satu codebase. Bahasa yang dipakai adalah Dart. UI Flutter dirender menggunakan engine sendiri (Skia/Impeller), bukan widget native, sehingga tampilannya konsisten di semua device.	EndeavourOS (Arch)	`yay -S flutter`
Dart SDK	Bahasa pemrograman yang dipakai Flutter. Dart mirip Java/JavaScript tapi lebih ringan. Lu tidak perlu master Dart — AI yang nulis kodenya, lu yang merakit.	Bundled dengan Flutter	`(otomatis saat install Flutter)`
Android Studio	IDE resmi Google yang HANYA dipakai untuk dua hal: menginstall Android SDK dan membuat/menjalankan Emulator HP Android. Setelah itu, tutup — tidak perlu dibuka lagi.	EndeavourOS (Arch)	`yay -S android-studio`
Cursor IDE	Code editor AI-powered (VS Code versi dewa) tempat lu nulis semua kode Flutter. Ctrl+L untuk memanggil AI, kasih screenshot desain, minta AI buatkan kode Flutter-nya.	EndeavourOS (Arch)	`yay -S cursor-bin`
Riverpod	Package State Management terbaik untuk Flutter. Ini yang membuat angka saldo di layar otomatis berubah saat tombol "Bayar" ditekan, tanpa perlu reload halaman.	Flutter Package	`flutter pub add flutter_riverpod`
Dio	HTTP client untuk Flutter. Ini yang dipakai untuk memanggil API Arienal dari dalam aplikasi — kirim request, terima response JSON, tampilkan di layar.	Flutter Package	`flutter pub add dio`
go_router	Package navigasi (routing) untuk berpindah antar halaman di Flutter. Lebih powerful dari Navigator bawaan Flutter, mendukung deep link.	Flutter Package	`flutter pub add go_router`
flutter_secure_storage	Menyimpan data sensitif (JWT token dari Arienal) secara aman di penyimpanan terenkripsi device. JANGAN simpan token di SharedPreferences biasa.	Flutter Package	`flutter pub add flutter_secure_storage`
local_auth	Package untuk autentikasi biometrik (sidik jari/Face ID). Dipakai untuk fitur 2FA sebelum 1-Tap Approve — user harus scan jari dulu sebelum uang "disapu".	Flutter Package	`flutter pub add local_auth`
Postman	Alat untuk mengetes API Arienal sebelum diintegrasikan ke Flutter. Pakai Postman untuk	EndeavourOS (Arch)	`yay -S postman-bin`

	memahami format request/response JSON dari setiap endpoint.		
Figma (view only)	Platform desain UI. Lu tidak perlu desain dari nol — Nabil yang desain. Lu hanya perlu bisa membaca/screenshot desain Figma dan menerjemahkannya ke kode Flutter.	Browser / Web	Akses via figma.com

💡 **MENTAL MODEL TOOLS**

Flutter = Kanvas dan kuas. Dart = Bahasa untuk instruksikan kuas. Android Studio = Tukang kunci yang buka pintu ke emulator. Cursor IDE = Asisten pelukis AI yang bisa bikin sketsa dari foto. Riverpod = Sistem saraf yang menghubungkan semua bagian layar. Dio = Kurir yang ambil data dari server Arienal.

🛠️ TAHAP 1 — FASE SIMULASI KOTOR

Sekarang – 30 April 2025 · Bangun dan slicing UI pakai data bohongan, rakit state management, siap sambungin ke API di Fase 2.

LANGKAH 1 — Setup Workspace di EndeavourOS (Arch Linux)

Sebelum nulis satu baris kode Flutter, semua senjata harus terinstall. Karena kamu pakai Arch Linux (EndeavourOS), semua instalasi lewat terminal menggunakan `yay` (AUR helper). Jalankan semua perintah ini secara berurutan.

1

Update System & Install Flutter SDK

Buka terminal (biasanya Ctrl+Alt+T atau lewat app launcher) dan jalankan:

```
# Update package database
sudo pacman -Syu

# Install yay jika belum ada (AUR helper)
sudo pacman -S --needed base-devel git
git clone https://aur.archlinux.org/yay.git
cd yay && makepkg -si

# Install Flutter, Android Studio, dan Cursor sekaligus
yay -S flutter android-studio cursor-bin

# Cek Flutter terinstall
flutter --version

# Jalankan Flutter doctor untuk cek status environment
flutter doctor
```

⚠️ **PERHATIAN: flutter doctor**

Setelah install, jalankan flutter doctor. Akan ada beberapa item yang belum centang hijau — no
Yang paling penting: pastikan "Flutter" dan "Android toolchain" sudah ✓. Item lain bisa diabaika

2 Setup Android Studio (Sekali Saja)

Buka Android Studio (bukan untuk coding — hanya untuk install SDK dan buat emulator). Ini hanya dilakukan SATU KALI:

1	Buka Android Studio → klik "More Actions" → SDK Manager
2	Di tab "SDK Platforms" → centang Android 13.0 (API 33) atau versi terbaru
3	Di tab "SDK Tools" → centang Android SDK Build-Tools, Android Emulator, Android SDK Platform-Tools
4	Klik Apply → tunggu download selesai
5	Kembali ke menu utama → "More Actions" → Virtual Device Manager
6	Klik "+ Create Device" → pilih Pixel 6 → pilih Android 13 System Image
7	Finish → klik tombol Play (▶) di emulator untuk memastikan bisa nyala
8	Tutup Android Studio — TIDAK PERLU dibuka lagi setelah ini

💡 **Kenapa Android Studio harus ditutup?**

Android Studio sangat berat di RAM (bisa 2-3GB). Karena lu ngoding di Cursor IDE, Android St hanya dipakai sebagai "alat bantu" install SDK dan emulator. Setelah SDK terinstall, Cursor + F CLI yang jalan — Android Studio tidak diperlukan lagi.

3 Setup Cursor IDE & Flutter Extension

Cursor adalah IDE utama lu. Setelah install, lakukan konfigurasi ini:

```
# Buka Cursor dari terminal
cursor

# Atau cari di app launcher: "Cursor"
```

Di dalam Cursor, install extension berikut (Ctrl+Shift+X):
→ Flutter (by Dart Code) — wajib, syntax highlighting + hot reload
→ Dart (by Dart Code) — wajib, language support
→ Pubspec Assist — opsional, bantu tambah package lebih mudah

→ Error Lens — opsional, tampilkan error langsung di baris kode

🔑 **CARA PAKAI AI DI CURSOR**

Tekan Ctrl+L untuk membuka panel chat AI. Di sini lu bisa: (1) Upload screenshot desain Figma ketik "Buatkan kode Flutter untuk layout ini", (2) Paste error dari terminal lalu ketik "Jelaskan da perbaiki error ini", (3) Pilih kode → Ctrl+K untuk edit/refactor dengan instruksi.

4

Buat Project Flutter Pertama

Buka terminal baru (venv tidak diperlukan di Flutter — pakai pub), lalu jalankan:

```
# Buat project Flutter baru
flutter create lunas_app

# Masuk ke folder project
cd lunas_app

# Buka di Cursor IDE
cursor .

# Nyalakan emulator dulu (dari terminal terpisah)
flutter emulators --launch <nama_emulator>

# Jalankan app ke emulator
flutter run

# Kalau berhasil, layar emulator menampilkan "Flutter Demo Home Page"
# Ini tanda Flutter sudah jalan sempurna!
```

🎯 **OUTPUT LANGKAH 1**

Emulator Android nyala. App default Flutter (counter) muncul di layar emulator. Flutter doctor tic error merah. Cursor IDE terbuka dengan project lunas_app dan syntax highlighting berjalan (ko berwarna). Kalau semua ini terpenuhi: Workspace siap tempur.

LANGKAH 2 — Memahami Anatomi Flutter (Fondasi Visual)

Sebelum slicing UI, lu harus paham 5 konsep widget fundamental Flutter. Ini adalah "kosakata" dasar yang lu pakai untuk merakit SEMUA layar LUNAS. Buka Cursor, buat file baru, dan praktikkan satu per satu.

Widget	Analogi	Kapan Dipakai di LUNAS
Scaffold	Kerangka bangunan — dinding, atap, lantai	Setiap halaman LUNAS wajib punya Scaffold. Di dalamnya ada AppBar, body, floatingActionButton.

Column	Susun kotak-kotak dari atas ke bawah	Dashboard: nama user di atas, angka saldo di tengah, daftar utang di bawah — semua tersusun vertikal.
Row	Susun kotak-kotak dari kiri ke kanan	Satu item tagihan utang: ikon di kiri, nama utang di tengah, angka rupiah di kanan.
Card	Kertas catatan dengan bayangan (shadow)	Card besar di atas dashboard menampilkan Net-Worth. Card kecil untuk setiap item tagihan.
ListView.builder	Daftar panjang yang bisa di-scroll, dibuat otomatis dari data	Daftar riwayat transaksi dan daftar semua tagihan utang — tidak mau render 100 item sekaligus, harus lazy load.
Container	Kotak serba guna dengan ukuran, warna, padding	Alert Nudging AI — kotak merah/kuning yang muncul berisi pesan teguran dari Ibnul.
Stack	Tumpuk widget di atas satu sama lain	Avatar user dengan badge notifikasi di sudut kanan atas.
GestureDetector / InkWell	Tombol tak kasat mata — bisa tap apapun	Semua elemen yang bisa diklik: card tagihan, tombol custom, area swipe.

💬

PROMPT KE CURSOR AI — Latihan Widget Dasar

Copy-paste prompt ini ke Cursor AI (Ctrl+L) untuk langsung belajar sambil praktik:

```
"Buatkan file Flutter bernama latihan_widget.dart yang berisi:
 1. Scaffold dengan AppBar bertuliskan "Belajar Flutter - LUNAS"
 2. Body berisi Column dengan:
    - Card besar berisi Row: teks "Total Utang" di kiri, angka "Rp 4.300.0
      di kanan dengan warna merah tebal
    - SizedBox(height: 16) sebagai jarak
    - Teks "Daftar Tagihan" tebal
    - ListView dengan 3 item dummy: ShopeePay Later, AdaKami, Kredivo
      masing-masing tampilkan nama dan angka utang
 3. Gunakan dummy data statis (hardcode). Tidak perlu API dulu."
```

💡 **CARA BELAJAR YANG BENAR**

Jangan hanya copy-paste kode AI dan jalankan. Baca setiap baris. Kalau ada yang tidak pahar highlight baris itu → Ctrl+L → ketik "Jelaskan kode ini baris per baris dalam bahasa Indonesia". Cursor akan menjelaskannya. Pemahaman > copy-paste.

LANGKAH 3 — UI Slicing: Menerjemahkan Desain ke Kode

Minta Nabil kirimkan screenshot atau akses view-only Figma desain LUNAS. Target bulan April: slicing 3 layar utama menggunakan dummy data statis. Belum ada koneksi ke API — semua angka masih bohongan.

🏠 Net-Worth Dashboard	⚡ 1-Tap Approve Pop-Up	🔔 Nudging Alert Widget
Halaman utama. Card besar atas: Saldo vs Total Utang. Progress bar bebas utang. ListView tagihan aktif. Tombol floating "1-Tap Approve".	Modal Bottom Sheet yang muncul dari bawah saat tombol "Bayar" ditekan. Tampilkan rincian: berapa yang akan disapu, ke utang mana, tanda biometrik.	Komponen UI (bukan full page) berupa banner merah/kuning di atas dashboard. Berisi icon peringatan, judul teguran, dan teks pesan dari AI Ibnul.

1

Layar 1: Dashboard Net-Worth

Buat file baru `lib/screens/dashboard_screen.dart`. Minta Cursor AI dengan prompt:

```
"Buatkan Flutter screen untuk Dashboard Net-Worth aplikasi LUNAS.
 Spesifikasi layout:
 - Scaffold dengan AppBar: foto profil user (CircleAvatar), nama user, ico
notifikasi
 - Card utama (shadow tebal, rounded 16px) berisi:
    · Teks label "Net-Worth Kamu" kecil abu-abu
    · Angka besar "Rp 12.500.000" warna hijau tebal
    · Row berisi dua mini-card: "Saldo" Rp 15.000.000 dan "Total Utang" Rp
2.500.000 merah
    · LinearProgressIndicator untuk progress bebas utang (value: 0.42)
    · Teks "42% Menuju Bebas Utang — estimasi 8 bulan lagi"
 - Judul section "Tagihan Aktif" tebal
 - ListView.builder dengan 3 dummy item tagihan, masing-masing Card kecil
    · Row: nama utang (kiri), jumlah utang merah (kanan)
    · Row: badge jenis (paylater/pinjol) dan APR % (kanan)
 - FloatingActionButton bertuliskan "⚡ 1-Tap Sweep"
 Gunakan dummy data statis. Warna tema: ungu (#6C63FF) dan putih."
```

🎯 **OUTPUT LAYAR 1**

Layar dashboard bisa dibuka di emulator. Tampil Card Net-Worth dengan angka dummy. ListVie
menampilkan 3 tagihan dummy bisa di-scroll. FAB "1-Tap Sweep" terlihat di kanan bawah. Sem
masih hardcode — belum perlu API.

2

Layar 2: Modal 1-Tap Approve (Bottom Sheet)

Buat file `lib/widgets/approve_bottom_sheet.dart`. Ini adalah komponen yang muncul dari bawah saat tombol "⚡ 1-Tap Sweep" di-tap:

```
"Buatkan Flutter widget untuk showModalBottomSheet yang muncul saat tombol
 1-Tap Sweep ditekan di dashboard LUNAS. Spesifikasi:
 - Shape: rounded top corners radius 24px
 - Handle bar abu-abu di atas (pill shape)
 - Judul besar: "Konfirmasi Auto-Sweep"
 - Card berisi rincian sweep:
```

```
    · "Dana yang akan disapu: Rp 500.000"
    · "Diprioritaskan ke: AdaKami Pinjol (APR 70%)"
    · "Setelah sweep, sisa utang: Rp 300.000"
    · Teks hijau kecil: "Kamu hemat bunga Rp 17.500 bulan ini!"
  - Row dua tombol: "Batal" (outlined, abu-abu) dan "Konfirmasi ✓" (filled
  - Teks kecil di bawah: "🔒 Akan diminta verifikasi biometrik"
  Gunakan dummy data statis. Bottom sheet harus bisa di-dismiss dengan swip
down."
```

💡 **KENAPA BOTTOM SHEET, BUKAN FULL PAGE?**

Bottom sheet adalah pola UX yang tepat untuk konfirmasi cepat. User bisa lihat context dashbo belakangnya, dan aksi cepat bisa dilakukan tanpa meninggalkan halaman. Ini yang membuat fi "1-Tap" terasa instantaneous — psychological design yang bagus untuk hackathon demo.

3

Layar 3: Widget Alert Nudging AI

Buat file `lib/widgets/nudging_alert.dart`. Ini adalah komponen banner yang muncul di atas dashboard ketika AI Ibnul mendeteksi perilaku boros:

```
"Buatkan Flutter widget NudgingAlert yang bisa ditempatkan di atas
 dashboard LUNAS. Spesifikasi:
 - Container dengan rounded corners 12px
 - Tiga level severity dengan warna berbeda:
    · warning (kuning #F5A623): peringatan ringan
    · danger (oranye #E67E22): peringatan sedang
    · critical (merah #E74C3C): peringatan serius
 - Layout Row: icon di kiri (⚠️/🔥/🚨), konten di tengah, tombol X di kana
 - Konten: judul tebal kecil + teks pesan (dari parameter)
 - Animasi: slide-in dari atas saat widget muncul (AnimationController)
 - Bisa di-dismiss (tap X → hilang dengan animasi fade out)
 - Parameter: String title, String message, String severity",

// Contoh pemakaian di dashboard:
NudgingAlert(
  title: "Pengeluaran Konsumtif Melonjak!",
  message: "Lu udah belanja Rp 1.2jt hari ini.",
  severity: "danger",
)
```

🎯 **OUTPUT LAYAR 3**

Widget NudgingAlert bisa dipanggil di mana saja. Warna berubah sesuai severity. Animasi slide atas terlihat smooth. Tombol X berfungsi dismiss dengan fade out. Ini akan diisi teks real dari A Fase 2.

LANGKAH 4 — Menghidupkan Aplikasi dengan State Management (Riverpod)

Sampai Langkah 3, semua data di layar masih hardcode (statis). Langkah ini mengubah aplikasi jadi "hidup" — angka di layar bereaksi saat tombol ditekan. Ini adalah perbedaan antara mockup dan aplikasi sungguhan.

APA ITU STATE MANAGEMENT?

Bayangkan saldo kamu tertulis di papan tulis besar di atas layar HP. State management adalah "otak" yang memutuskan: kapan tulisan di papan berubah, dan memberitahu semua bagian layar untuk update tampilannya. Tanpa state management, tombol "Bayar" ditekan → tidak ada yang berubah di layar. Dengan Riverpod, tombol "Bayar" ditekan → saldo otomatis berkurang → ListView tagihan auto-refresh → progress bar bergerak — semua terjadi dalam milidetik.

1

Setup Riverpod & Buat Provider Pertama

Install Riverpod dan buat state management untuk dashboard:

```
# Install package Riverpod
flutter pub add flutter_riverpod

# Juga install riverpod annotation untuk code generation (opsional tapi
recommended)
flutter pub add riverpod_annotation
flutter pub add --dev riverpod_generator build_runner
```

Buka Cursor → Ctrl+L → paste prompt ini:

```
"Buatkan implementasi Riverpod untuk state management Dashboard LUNAS.
 Buat file lib/providers/dashboard_provider.dart yang berisi:

 1. Model class DashboardState:
    - double saldo (default 15000000)
    - double totalUtang (default 4300000)
    - double netWorth (computed: saldo - totalUtang)
    - double progressBebasUtang (computed: 0.0 sampai 1.0)
    - List<TagihanModel> daftarTagihan

 2. Model class TagihanModel:
    - String id, namaUtang, jenis (paylater/pinjol)
    - double totalUtang, sukuBungaApr

 3. Class DashboardNotifier extends StateNotifier<DashboardState>:
    - Method bayarTagihan(String tagihanId, double jumlah):
      kurangi saldo, kurangi utang tagihan terkait, update state
    - Method tambahTagihan(TagihanModel tagihan): tambah ke list

 4. Provider: dashboardProvider = StateNotifierProvider

 Gunakan dummy data untuk inisialisasi. Sertakan contoh penggunaan
 di DashboardScreen dengan ref.watch dan ref.read."
```

2

Sambungkan Provider ke UI Dashboard

Setelah provider dibuat, update DashboardScreen untuk menggunakan data real dari provider (bukan hardcode):

```
// lib/screens/dashboard_screen.dart
// Ubah class dari StatefulWidget menjadi ConsumerWidget (Riverpod)

class DashboardScreen extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // Watch state — layar otomatis rebuild saat state berubah
    final state = ref.watch(dashboardProvider);

    return Scaffold(
      body: Column(
        children: [
          // Sekarang pakai data dari state, bukan hardcode
          NetWorthCard(netWorth: state.netWorth, saldo: state.saldo),
          LinearProgressIndicator(value: state.progressBebasUtang),
          ListView.builder(
            itemCount: state.daftarTagihan.length,
            itemBuilder: (ctx, i) => TagihanCard(
              tagihan: state.daftarTagihan[i],
              // Tombol bayar memanggil notifier
              onBayar: () => ref.read(dashboardProvider.notifier)
                .bayarTagihan(state.daftarTagihan[i].id, 500000),
            ),
          ),
        ],
      ),
    );
  }
}
```

🎯 **OUTPUT LANGKAH 4**

Tekan tombol "Bayar" di salah satu tagihan → angka saldo berkurang real-time tanpa reload → progress bar bergerak → list tagihan terupdate. Ini adalah "wow moment" pertama: aplikasi yan Tunjukkan ini ke Nabil dan tim untuk konfirmasi sebelum lanjut ke Fase 2.

LANGKAH 5 — Latihan HTTP Request dengan Dummy API

Di Fase 2, lu akan connect ke API Arienal yang real. Tapi sebelum itu, pelajari cara Flutter mengambil data dari internet menggunakan package Dio dengan URL dummy dari JSONPlaceholder atau Postman Mock Server.

1

Install Dio & Buat API Service

Install Dio dan buat layer service untuk HTTP request:

```
# Install Dio
```

```
flutter pub add dio

# Install juga package untuk environment variables
flutter pub add flutter_dotenv
```

Buka Cursor → Ctrl+L → paste prompt ini:

```
"Buatkan file lib/services/api_service.dart untuk LUNAS yang berisi:

 1. Class ApiService dengan Dio instance
 2. Base URL diambil dari environment variable (pakai dotenv)
 3. Interceptor untuk otomatis attach JWT token ke setiap request
 4. Method-method berikut (semua async, return Future):
    - getDashboard(int userId) → GET /api/v1/dashboard/{userId}
    - getTagihan(int userId) → GET /api/v1/tagihan/{userId}
    - hitungAvalanche(Map data) → POST /api/v1/hitung-avalanche
    - getNudging(Map data) → POST /api/v1/nudging
    - approveSwee(Map data) → POST /api/v1/sweep/approve
 5. Error handling: catch DioException, return null atau throw custom erro

 Untuk testing sekarang, pakai base URL: https://jsonplaceholder.typicode.
 (ganti nanti ke URL Arienal di Fase 2)"
```

📌 **KENAPA DIPISAH DI services/?**

Memisahkan HTTP calls ke file service terpisah (bukan langsung di UI) adalah best practice. Ka
API berubah atau format JSON berubah, lu hanya perlu edit SATU file (api_service.dart), bukar
ganti di puluhan file UI. Ini yang disebut "separation of concerns".

⚔️ TAHAP 2 — FASE TEMPUR MVP

01 Mei – 04 Juni 2025 · Sambungkan layar Flutter ke API Arienal. Aplikasi beralih dari dummy data ke data real. Masuk ke production.

APA YANG BERUBAH DI FASE INI?

Di Tahap 1, semua data masih bohongan — hardcode di kode Flutter. Di Tahap 2, angka Rp 5.000.000 dummy itu diganti dengan data nyata dari database Arienal. Layar HP lu sekarang menjadi jendela ke data real user.

A

Terima Handoff dari Arienal: Pahami Swagger UI

Arienal akan kirimkan link Swagger UI (biasanya `localhost:8000/docs` atau URL production). Ini adalah "manual API" — panduan semua endpoint yang bisa dipanggil Flutter.

Endpoint Arienal	Method	Yang Diperlukan di Flutter

/api/v1/auth/login	POST	Halaman login — kirim email+password, terima JWT token
/api/v1/tagihan/{user_id}	GET	Dashboard — ambil daftar semua tagihan aktif user
/api/v1/hitung-avalanche	POST	Sebelum bottom sheet — hitung rencana sweep optimal
/api/v1/nudging	POST	Dashboard — ambil pesan teguran AI untuk banner NudgingAlert
/api/v1/sweep/approve	POST	Tombol konfirmasi di bottom sheet — eksekusi pembayaran
/api/v1/dashboard/{user_id}	GET	Dashboard — ambil Net-Worth, saldo, progress bebas utang

B

Ganti Dummy Data dengan API Calls Real

Ini adalah momen paling kritis Fase 2. Satu per satu, ganti data hardcode di provider dengan pemanggilan API:

```
// SEBELUM (Fase 1 — dummy data):
class DashboardNotifier extends StateNotifier<DashboardState> {
  DashboardNotifier() : super(DashboardState(
    saldo: 15000000,  // ← HARDCODE
    totalUtang: 4300000,  // ← HARDCODE
    daftarTagihan: dummyTagihan,  // ← HARDCODE
  ));
}

// SESUDAH (Fase 2 — data real dari Arienal):
class DashboardNotifier extends StateNotifier<AsyncValue<DashboardState>>
  final ApiService _api;
  DashboardNotifier(this._api) : super(const AsyncValue.loading());

  Future<void> loadDashboard(int userId) async {
    try {
      final data = await _api.getDashboard(userId);  // ← CALL API ARIENAL
      state = AsyncValue.data(DashboardState.fromJson(data));
    } catch (e) {
      state = AsyncValue.error(e, StackTrace.current);
    }
  }
}
```

⚠️ **HANDLING LOADING & ERROR STATE**

Saat memanggil API, ada tiga kondisi: loading (data belum datang), data (sukses), error (jaring server error). WAJIB handle ketiganya di UI. Gunakan AsyncValue dari Riverpod: tampilkan

CircularProgressIndicator saat loading, tampilkan data saat sukses, tampilkan pesan error yang
(bukan stack trace) saat gagal.

C

Implementasi Autentikasi JWT & Biometrik

Setelah API terhubung, implementasi sistem login lengkap dengan keamanan biometrik untuk fitur 1-Tap Approve:

```
// Alur autentikasi lengkap LUNAS:

// 1. User input email + password di LoginScreen
// 2. Flutter POST ke /api/v1/auth/login milik Arienal
// 3. Arienal kembalikan JWT token
// 4. Simpan token di flutter_secure_storage (bukan SharedPreferences!)
// 5. Setiap request ke API selanjutnya: attach token di header

// Simpan token:
final storage = FlutterSecureStorage();
await storage.write(key: "jwt_token", value: responseData["token"]);

// Baca token untuk request berikutnya:
final token = await storage.read(key: "jwt_token");

// Biometrik sebelum 1-Tap Approve:
final localAuth = LocalAuthentication();
final didAuthenticate = await localAuth.authenticate(
  localizedReason: "Verifikasi identitas untuk konfirmasi sweep",
  options: AuthenticationOptions(biometricOnly: true),
);
if (didAuthenticate) {
  // Baru panggil API sweep/approve
  await apiService.approveSwipe(sweepData);
}
```

D

Integrasi Nudging Alert dengan AI Ibnul

Teks teguran dari AI Ibnul akan dikirim lewat API Arienal di endpoint `/api/v1/nudging`. Tugasmu memastikan teks itu muncul dengan desain yang "galak tapi keren" di widget NudgingAlert:

```
// Panggil API nudging setiap kali ada transaksi baru
Future<void> checkNudging(int userId) async {
  final transaksiTerbaru = state.value?.transaksiTerbaru;

  final nudgingData = await _api.getNudging({
    "user_id": userId,
    "pengeluaran": transaksiTerbaru,
    "utang_aktif": state.value!.totalUtang,
```

```
      "gaji": state.value!.estimasiGaji,
    });

    if (nudgingData != null) {
      // Update state untuk tampilkan banner
      state = state.whenData((s) => s.copyWith(
        nudgingMessage: nudgingData["nudging_message"],
        nudgingSeverity: _parseSeverity(nudgingData["severity"]),
        showNudging: true,
      ));
    }
  }
```

📌 **KOORDINASI DENGAN IBNUL**

Sebelum integrasi, pastikan format response JSON dari /api/v1/nudging sudah disepakati. Mini harus ada: "nudging_message" (string teks teguran), "severity" (warning/danger/critical), dan "kategori_teguran" (pengeluaran/utang/pola). Buat dokumen kontrak format data di Notion atau chat tim.

📋 PETA ENDPOINT API — REFERENSI LENGKAP FLUTTER

Semua API call yang harus diimplementasikan di Flutter. Gunakan sebagai checklist integrasi.

Method	Status	Endpoint	Yang Dilakukan Flutter
POST	Wajib Tahap 1	`/api/v1/auth/register`	Halaman registrasi — kirim nama, email, password → terima user object
POST	Wajib Tahap 1	`/api/v1/auth/login`	Halaman login — kirim email+password → terima JWT token, simpan di secure storage
GET	Wajib Tahap 1	`/api/v1/tagihan/{user_id}`	Dashboard — ambil list semua tagihan aktif user untuk ditampilkan di ListView
POST	Wajib Tahap 1	`/api/v1/tagihan`	Form tambah tagihan — kirim data utang baru (nama, jumlah, APR, jenis)
POST	🔥 Inti LUNAS	`/api/v1/hitung-avalanche`	Dipanggil sebelum bottom sheet — hitung urutan optimal pembayaran utang
POST	🔥 Inti LUNAS	`/api/v1/nudging`	Dipanggil setelah transaksi — ambil pesan teguran AI untuk NudgingAlert banner
POST	🔥 Inti LUNAS	`/api/v1/sweep/calculate`	Hitung berapa dana di atas buffer yang bisa di-sweep sekarang

POST	Tahap 2	/api/v1/sweep/approve	1-Tap Approve: setelah biometrik → konfirmasi eksekusi pembayaran real
GET	Tahap 2	/api/v1/dashboard/{user_id}	Ambil data agregat: Net-Worth, saldo, total utang, progress, estimasi bebas utang
GET	Kapan saja	/health	Ping server — cek apakah API Arienal masih online sebelum semua call lainnya

🔥 CHECKLIST RIZKI — DEADLINE 30 APRIL

Semua item ini harus selesai sebelum masuk Fase Tempur. Centang satu per satu saat selesai.

🛠️ SETUP & TOOLS

☐ Flutter SDK berhasil terinstall di EndeavourOS. Verifikasi: flutter --version dan flutter doctor tidak ada error merah di bagian Flutter dan Android toolchain.

☐ Android Studio berhasil install Android SDK dan emulator Pixel 6 (API 33) bisa dinyalakan. Setelah itu Android Studio DITUTUP.

☐ Cursor IDE terinstall dan extension Flutter + Dart aktif. Syntax kode Flutter berwarna (syntax highlighting jalan).

☐ Project Flutter pertama berhasil dibuat dengan flutter create lunas_app dan app default bisa dijalankan ke emulator (muncul counter app).

📐 PEMAHAMAN WIDGET DASAR

☐ Bisa membedakan secara visual dan logika: Column (vertikal), Row (horizontal), Stack (tumpuk). Bisa jelaskan ke Nabil dengan kata-kata sendiri.

☐ Berhasil membuat latihan_widget.dart dengan Card, ListView.builder, dan minimal 3 widget berbeda.

🎨 UI SLICING

☐ Layar 1 (Dashboard Net-Worth) berhasil di-slice: Card Net-Worth, progress bar, ListView tagihan, dan FAB 1-Tap Sweep tampil di emulator dengan data dummy.

☐ Layar 2 (Modal Bottom Sheet 1-Tap Approve) berhasil: muncul dari bawah saat FAB ditekan, bisa di-dismiss dengan swipe, dua tombol (Batal & Konfirmasi) terlihat.

☐ Layar 3 (Widget NudgingAlert) berhasil: tampil dengan warna berbeda sesuai severity (warning/danger/critical), animasi slide-in jalan, tombol X bisa dismiss.

⚡ STATE MANAGEMENT

☐ Riverpod berhasil diinstall dan dikonfigurasi (ProviderScope di main.dart).

☐ Provider DashboardNotifier berhasil dibuat dengan state DashboardState (saldo, utang, list tagihan).

☐ DEMO WAJIB: Tombol "Bayar" di salah satu tagihan dummy ditekan → angka saldo di Card Net-Worth berkurang real-time → progress bar bergerak — TANPA reload halaman.

🌐 HTTP & API

☐ Package Dio berhasil diinstall. ApiService berhasil dibuat dengan method getDashboard, getTagihan, hitungAvalanche, getNudging.

☐ Berhasil test satu API call ke URL dummy (JSONPlaceholder) dari Flutter — response JSON berhasil di-parse dan ditampilkan di UI.

🤝 KOORDINASI TIM

☐ Sudah koordinasi dengan Arienal — sudah lihat/akses Swagger UI localhost:8000/docs dan memahami format endpoint utama.

☐ Sudah koordinasi dengan Ibnul — sudah sepakat format response JSON dari endpoint /nudging (ada field nudging_message, severity, kategori).

🤖 PROMPT MENTOR — UNTUK CLAUDE / GEMINI

Copy-paste prompt ini sebagai pesan pertama ke AI mentor kamu. Upload file Proposal ERRNOR sebelum paste prompt ini.

```
[SYSTEM OVERRIDE: LONG-TERM MENTORSHIP & CO-PILOT MODE INITIATED]

Halo. Mulai detik ini, kamu adalah Senior Mobile App Engineer (Flutter Expert)
& Lead Mentor saya. Nama saya Muhammad Rizki, dan peran saya adalah "The Face"
(Frontend & Mobile App Lead) untuk tim developer bernama ERRNOR.

Tolong BACA FILE PROPOSAL YANG SAYA LAMPIRKAN untuk memahami konteks penuh
aplikasi yang sedang kami bangun: LUNAS (aplikasi pelunasan utang cerdas).

[KONTEKS ENVIRONMENT SAYA]
- OS: EndeavourOS (Arch Linux)
- Package manager: yay dan pacman (BUKAN apt/brew)
- Level coding: Pemula, menggunakan gaya "Vibe Coding"
- Tim Backend (Arienal): sedang siapkan API FastAPI + PostgreSQL
- Tim AI (Ibnul): akan titip pesan nudging lewat API Arienal
- Ketua (Nabil): desain UI Figma + deploy ke GCP

[TECH STACK YANG DIGUNAKAN]
Framework: Flutter 3.x + Dart
State Management: Riverpod
HTTP Client: Dio
Auth Storage: flutter_secure_storage + local_auth (biometrik)
Navigation: go_router

[GAYA MENTORING YANG SAYA BUTUHKAN]
- Jadilah asisten interaktif pair-programming dari HARI INI sampai MVP selesai
- Bimbing saya selangkah demi selangkah
- Tunggu saya konfirmasi "BERHASIL" atau kirim error sebelum lanjut fase
- Semua perintah terminal harus menggunakan yay atau pacman (Arch Linux)
- Saya akan upload screenshot Figma → kamu buatkan kode Flutter-nya

[URUTAN FASE]
1. Setup Workspace EndeavourOS — install Flutter, Android Studio, Cursor
2. Hello LUNAS — project baru berjalan di emulator
3. Slicing UI — Dashboard, Bottom Sheet, NudgingAlert widget
4. State Management — Riverpod, tombol "Bayar" mengubah angka real-time
```

```
5. HTTP Request — Dio + ApiService, siap connect ke API Arienal

Jika kamu sudah baca proposal dan siap jadi mentor saya, balas dengan:
"System Ready, Rizki. Wajah LUNAS ada di tanganmu. Saya sudah menyesuaikan
environment dengan EndeavourOS. Mari mulai dari Fase 1.",
lalu tanya apakah saya siap dengan mengetik "GAS".
```

📋 **INSTRUKSI PEMAKAIAN PROMPT**

(1) Buka Claude atau Gemini Pro. (2) Klik ikon attachment/lampiran. (3) Upload file PDF Proposal Hackathon ERRNOR. (4) Copy-paste seluruh teks di kotak di atas. (5) Tekan Enter dan tunggu respons "System Ready". (6) Ketik "GAS" untuk mulai Fase 1.

You are the Face, Rizki.

Make it smooth, make it look expensive. 🚀

ERRNOR Team · 2025